

电机驱动软件框架架构设计文档

版本: v1.0

作者: 吴俊毅

日期: 2026-05-18

适用平台: STM32G474 + FreeRTOS + C++17 (ARM Compiler 6)

维护模式: 单人维护

目录

- 设计目标与约束
- 领域划分：通信域与控制域
- 五层架构
- 对象模型
- 控制域设计
- 通信域设计
- RTOS 角色与任务规划
- 设计模式清单
- 状态机设计
- 生命周期管理
- 版本管理
- 配置管理
- C/C++ 边界约定
- 编译与工具链
- 文件目录结构
- 代码量估算
- 渐进式迁移路线
- 维护纪律
- 架构评估

1. 设计目标与约束

1.1 设计目标

本架构为**单人维护的电机驱动固件框架**而设计，核心目标：

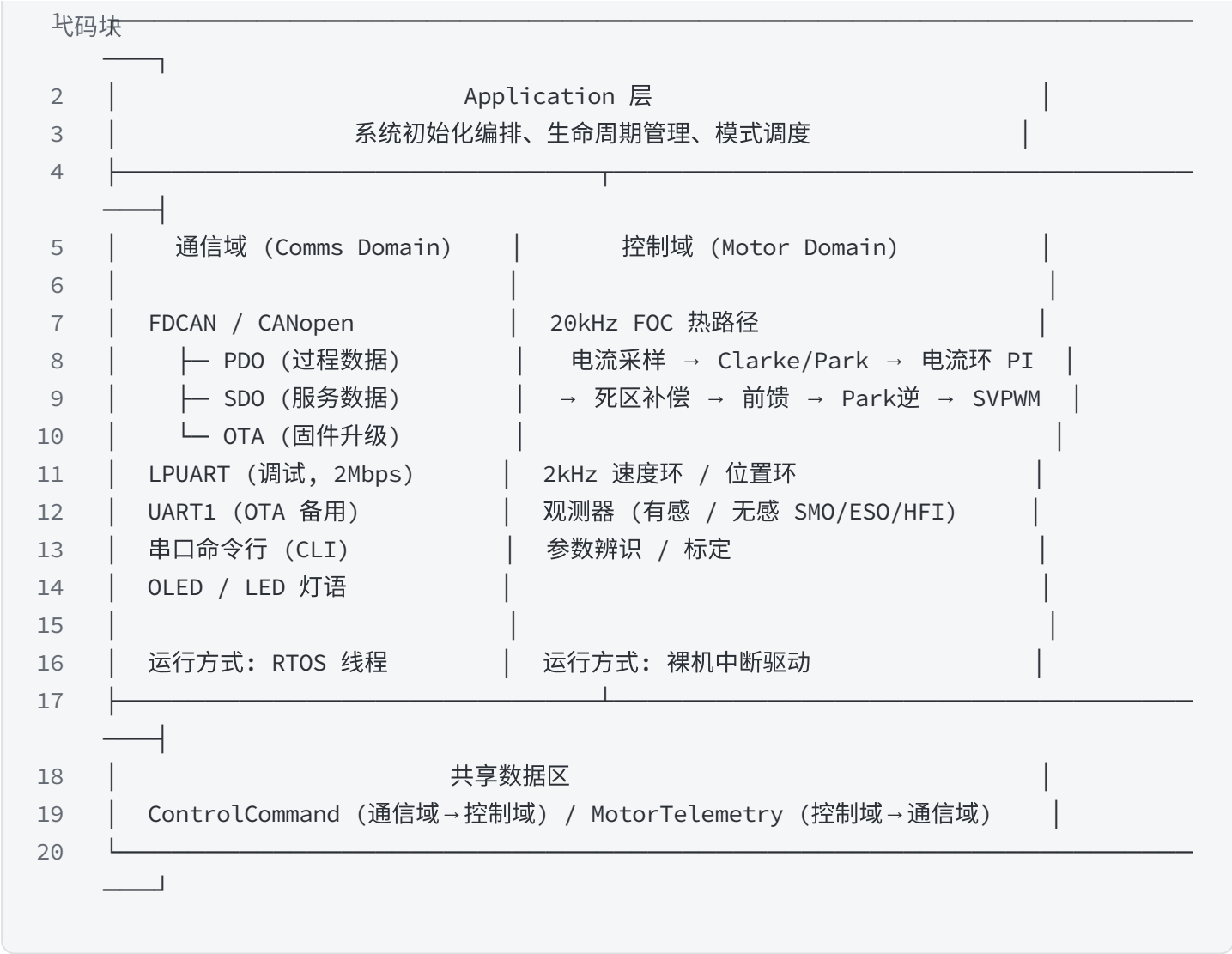
目标	说明
可替换	编码器、电流采样方式、功率级拓扑、控制策略（观测器/前馈/死区补偿）可独立替换，互不影响
可扩展	新增一种观测器或前馈策略只需新增一个文件，不动框架代码
可移植	换 MCU 或换 PCB 只修改 Platform 层和配置文件
可维护	单人能读懂全部代码，最长函数不超过 50 行，模块职责一句话说清
硬实时	20kHz FOC 中断路径零阻塞、零队列、零动态分配
保守	不使用堆分配、不使用 STL 容器、不使用智能指针、不使用异常和 RTTI

1.2 硬件约束

资源	数值
MCU	STM32G474RBT6
主频	170 MHz
片内 Flash	128 KB (Bootloader 64KB + APP 64KB)
SRAM	128 KB
FOC 中断频率	20 kHz (周期 50 μ s，约 8500 CPU 周期)
编译器	ARM Compiler 6 (Clang 14 级别)

2. 领域划分

整个固件分为**两个互不调用的域**，只通过共享数据区交换信息。

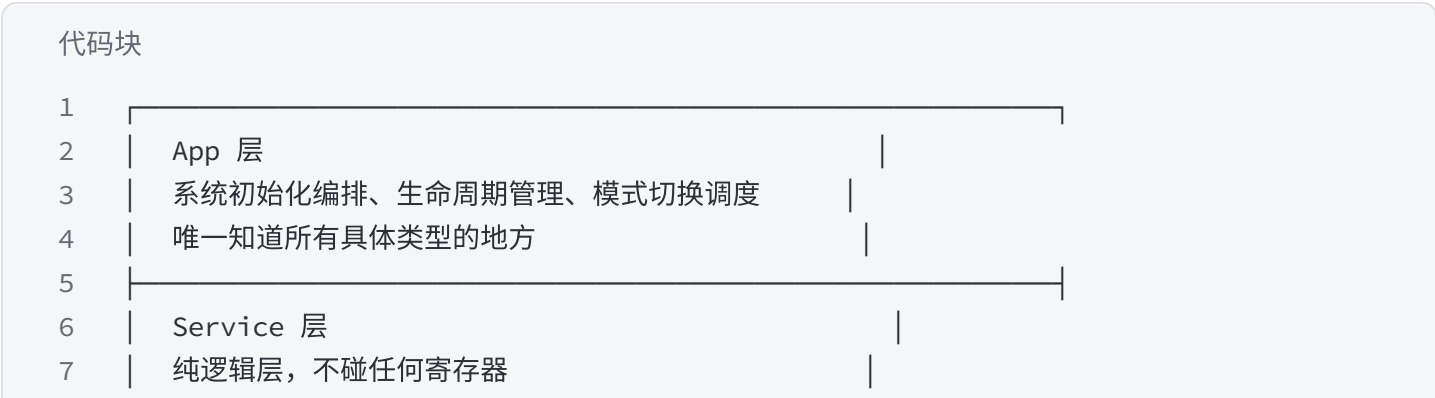


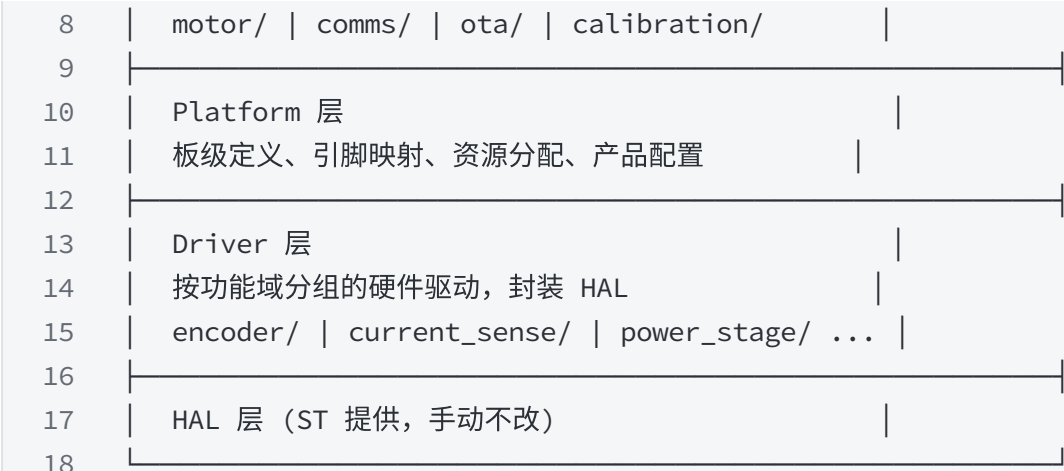
关键约束：

- 控制域**绝不调用**通信域的任何函数
- 通信域**绝不调用**控制域的任何函数
- 两个域有各自独立的初始化序列，在 App 层编排先后顺序

3. 五层架构

3.1 分层总览





3.2 依赖规则

- **单向依赖**：上层只依赖下一层的标准化接口
- **不跨层**：App 不能直接调 Driver，Service 不能直接调 HAL
- **接口隔离**：每层对外只暴露纯虚接口
- **下层不感知上层**：Driver 不知道 Service 的存在

3.3 各层职责

层	职责	不做什么
App	组装所有对象、编排初始化顺序、生命周期管理	不包含业务逻辑
Service	算法实现、协议处理、状态机	不碰寄存器、不调 HAL
Platform	引脚映射、硬件版本检测、产品配置	不包含驱动逻辑
Driver	封装 HAL、提供类型安全接口	不包含业务逻辑
HAL	ST 自动生成，裸寄存器操作	不改

4. 对象模型

每个功能模块 = **cfg + ctx + data + ops**。

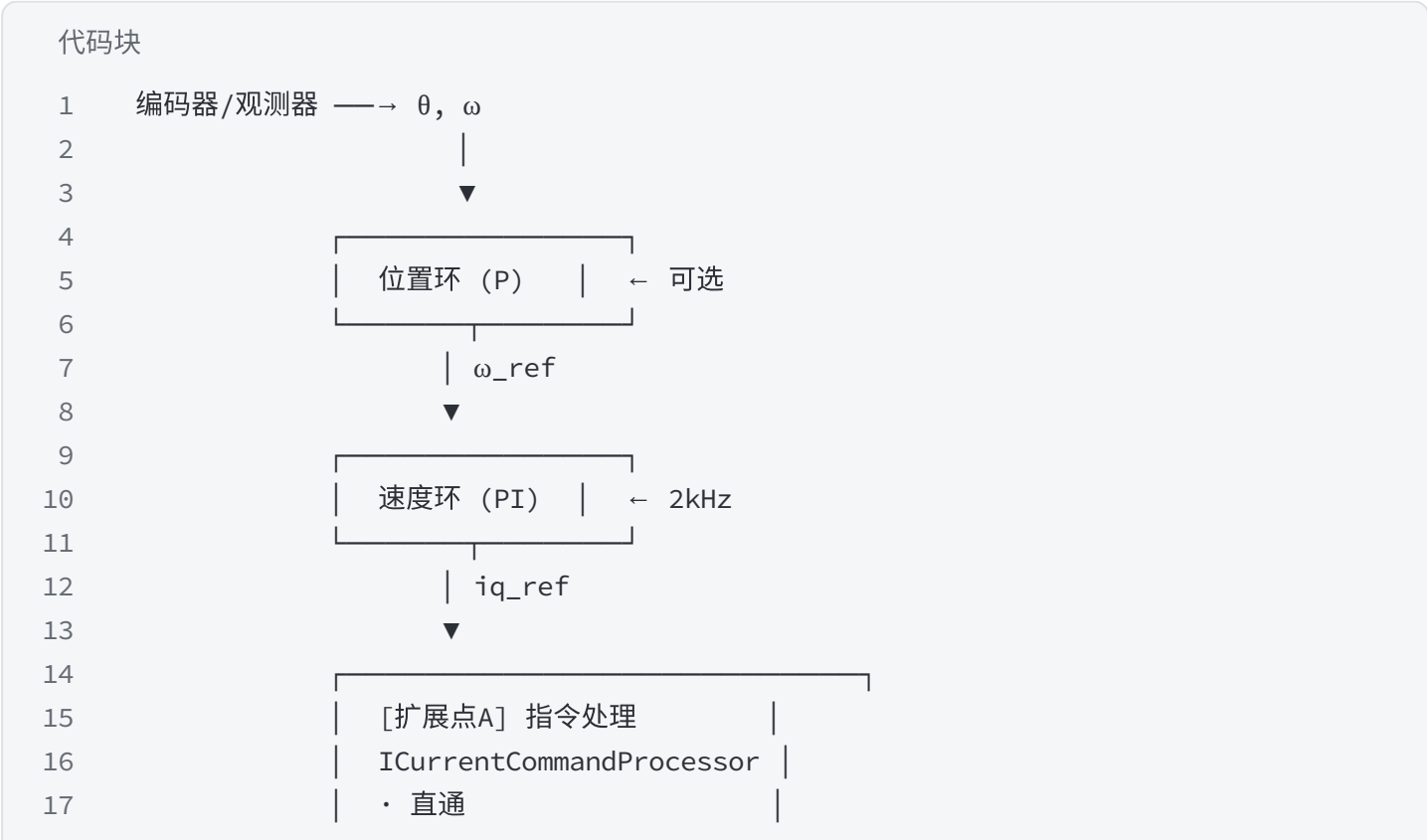
代码块

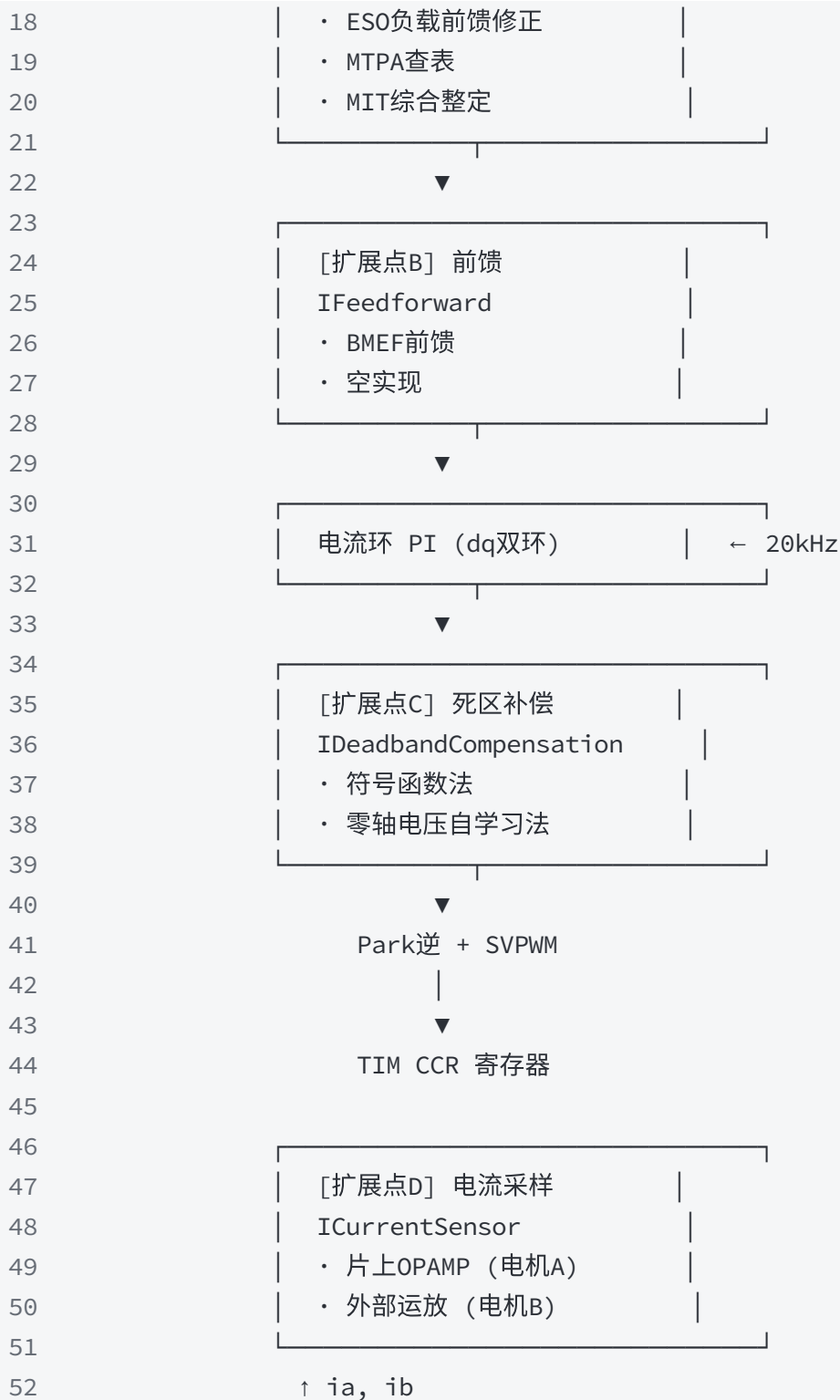
1

2	一个完整的对象	
3		
4	cfg	静态配置
5		引脚号、波特率、PI参数默认值
6		编译期确定或初始化时填入
7		运行期只读
8		
9	ctx	运行时上下文
10		状态机状态、计数器、时间戳
11		busy标志、错误计数
12		只有对象自己能修改
13		
14	data	当前处理的数据
15		电流、角度、速度、指令值
16		对外只读或通过原子操作访问
17		
18	ops	行为接口
19		init/start/process/stop
20		通过纯虚接口定义
21		

5. 控制域设计

5.1 数据流与扩展点





四个扩展点统一模式： 纯虚接口 + 初始化时注入具体实现 + 运行时不改指针。

5.2 核心接口定义

代码块

```

1  // ===== 观测器接口 =====
2  class IObserver {
3  public:
4      virtual ~IObserver() = default;

```

```

5     virtual bool init() = 0;
6     virtual void update(float ia, float ib, float ua, float ub) = 0;
7     virtual float getTheta() const = 0;
8     virtual float getOmega() const = 0;
9     virtual bool isConverged() const = 0;
10 };
11
12 // ===== 前馈接口 =====
13 class IFeedforward {
14 public:
15     virtual ~IFeedforward() = default;
16     virtual float computeUd(float id_ref, float iq_ref, float omega) = 0;
17     virtual float computeUq(float id_ref, float iq_ref, float omega) = 0;
18 };
19
20 // ===== 死区补偿接口 =====
21 class IDeadbandCompensation {
22 public:
23     virtual ~IDeadbandCompensation() = default;
24     virtual bool learn() = 0;
25     virtual void compensate(float ia, float ib, float& ua, float& ub) = 0;
26 };
27
28 // ===== 指令处理接口 =====
29 class ICurrentCommandProcessor {
30 public:
31     virtual ~ICurrentCommandProcessor() = default;
32     virtual void process(float& id_ref, float& iq_ref,
33                         float omega, float load_est) = 0;
34 };
35
36 // ===== 电流采样接口 =====
37 class ICurrentSensor {
38 public:
39     virtual ~ICurrentSensor() = default;
40     virtual bool init() = 0;
41     virtual void sample(float& ia, float& ib) = 0;
42     virtual float getDCLinkVoltage() const = 0;
43 };
44
45 // ===== 定时器接口 =====
46 class IPWMTimer {
47 public:
48     virtual ~IPWMTimer() = default;
49     virtual bool init(const PWMConfig& cfg) = 0;
50     virtual void setDuty(uint8_t motor_index, const PWMOutput& duty) = 0;
51     virtual void enable(uint8_t motor_index) = 0;

```

```

52     virtual void disable(uint8_t motor_index) = 0;
53     virtual uint8_t motorCount() const = 0;
54 };

```

5.3 MotorCore

代码块

```

1  class MotorCore {
2      ICurrentSensor& current_sensor_;
3      IPWMTimer&      pwm_timer_;
4      IObserver*      observer_ = nullptr;
5      IDeadbandCompensation* deadband_ = nullptr;
6      IFeedforward*   feedforward_ = nullptr;
7      ICurrentCommandProcessor* cmd_proc_ = nullptr;
8      // PI控制器、当前数据、状态 ...
9
10 public:
11     // 依赖注入
12     MotorCore(ICurrentSensor& cs, IPWMTimer& pwm, uint8_t idx);
13     void setObserver(IObserver* o) { observer_ = o; }
14     void setDeadbandComp(IDeadbandCompensation* d) { deadband_ = d; }
15     void setFeedforward(IFeedforward* f) { feedforward_ = f; }
16     void setCmdProcessor(ICurrentCommandProcessor* p) { cmd_proc_ = p; }
17
18     // 热路径 (20kHz TIM中断直接调用, 约40行)
19     void tick();
20     // 速度环 (2kHz)
21     void speedTick();
22 };

```

tick() 开销分析： 虚函数调用 $\times 5 \approx 10$ CPU周期，占 20kHz 周期 (8500 cycle) 的 **0.12%**。SVPWM 计算是其 50 倍，可忽略。

6. 通信域设计

6.1 CANopen 中断两级分流

代码块

```

1  FDCAN 接收中断
2  |—— 快速路径 (ISR 内完成, <2μs):
3  |    PDO → 直接写 ControlCommand 共享结构体

```

```
4      |      NMT → 直接改状态
5      |      全部 0(1) 内存写
6      └─ 慢速路径（入队列，RTOS 线程处理）：
7          SDO 分段传输 → 入 SPSC_RingBuffer → osSignal
```

6.2 共享数据结构

代码块

```
1  // 通信域 → 控制域
2  struct ControlCommand {
3      uint16_t controlword;          // 6040
4      int32_t  target_position;      // 607A
5      int32_t  target_velocity;      // 60FF
6      int16_t  target_torque;        // 6071
7      uint8_t  mode_of_operation;    // 6060
8      uint32_t timestamp;
9      bool     new_command;
10 };
11
12 // 控制域 → 通信域
13 struct MotorTelemetry {
14     int32_t  actual_position;        // 6064
15     int32_t  actual_velocity;        // 606C
16     int16_t  actual_torque;          // 6077
17     uint16_t statusword;             // 6041
18     uint8_t  mode_display;           // 6061
19     float    ia, ib, id, iq;
20     float    theta, omega;
21     uint16_t fault_code;
22 };
```

6.3 三种生产者-消费者模式

类型	场景	实现
SPSC 环形缓冲	CAN ISR → RTOS 线程	<code>SPSC_RingBuffer<T, N></code> ，无锁
双缓冲	FOC ISR 写遥测 → CANopen 读	<code>TelemetryBuffer</code> ，原子交换 write/read idx
原子请求	多方请求模式切换	<code>std::atomic<bool></code> + <code>std::atomic<Mode></code>

7. RTOS 角色

原则：RTOS 只做辅助，硬实时全部在裸机中断。

任务规划

代码块

```
1  中断优先级（高于所有任务）：
2    TIM_FOC      (20kHz):  prio 0 (最高)
3    ADC 注入完成:      prio 1
4    TIM_SPEED    (2kHz):  prio 2
5    FDCAN RX:      prio 3
6    UART / DMA TC:    prio 4
7    TIM_BASE     (RTOS心跳): prio 5
8
9  RTOS 任务：
10   Task_CANOpenProcess  prio 6,  1kHz,  1.5KB 栈
11   Task_CLI             prio 5,  事件驱动, 2KB 栈
12   Task_OTA             prio 5,  事件驱动, 2KB 栈
13   Task_OLED            prio 4,  20Hz,  1KB 栈
14   Task_LED             prio 3,  50Hz,  512B 栈
15   Task_Logger          prio 3,  事件驱动, 2KB 栈
```

RTOS 不能碰的事

- ✗ 在任务里调用 `MotorCore::tick()`
- ✗ 在任务里写 TIM CCR 寄存器
- ✗ 用互斥锁保护热路径数据
- ✓ 任务可以：读共享数据、处理 SDO、执行 OTA、刷新 OLED、打印日志

8. 设计模式

只用到这些，且用最简单形式：

模式	实现	位置
策略	纯虚接口 + 指针注入	IObserver / IFeedforward / IDeadbandCompensation / ICmdProcessor

生产者-消费者 (队列)	SPSC 环形缓冲	CAN ISR → SDO 线程
生产者-消费者 (最新值)	双缓冲 + atomic	FOC ISR → 遥测
生产者-消费者 (请求)	atomic flag	模式切换请求
外观	MotorCore 类	FOC 唯一入口
适配器	Driver 实现类	封装 HAL
空对象	NullFeedforward / NullDeadband	不使能时注入，消除 nullptr 检查
状态	enum class + switch	两层状态机

明确不用：工厂模式、单例、观察者、装饰器、命令模式、DI 容器。

9. 状态机设计

9.1 两层状态机

代码块

```
1  大状态机 (SystemModeManager)：管理权限和安全
2  IDLE → SETUP → CALIBRATION → ENABLED
3      ↓           ↓           ↓
4  FAULT ←-----|
5      ↓
6  EMERGENCY
7
8  小状态机 (MotorModeManager)：仅在 ENABLED 下有效
9  PP ⇔ PV ⇔ CSP ⇔ CSV ⇔ CT, HM
```

9.2 大状态机用转移矩阵

代码块

```
1  constexpr bool transition_table_[6][6] = {
2      //      IDLE  SETUP  CALIB  ENAB  FAULT  EMERG
3      /*IDLE*/ {1,    1,    1,    1,    0,    0},
4      /*SETUP*/ {1,    1,    0,    0,    0,    0},
5      /*CALIB*/ {1,    0,    1,    0,    1,    0},
6      /*ENAB*/  {1,    0,    0,    1,    1,    0},
7      /*FAULT*/ {1,    0,    0,    0,    1,    1},
8      /*EMERG*/ {0,    0,    0,    0,    0,    1},
```

```
9    };
```

9.3 小状态机用 switch + 条件

事件响应层只写数据不改状态；条件检查层（2kHz tick）读数据判断是否转移。状态转移只有条件检查层一个入口，消除并发冲突。

10. 生命周期管理

电机驱动固件生命周期保守。所有对象在初始化时创建，永不销毁，无动态分配。

代码块

```
1  power_on → HAL基础初始化 → GPIO (PB8保留为BOOT0)
2  → 外设初始化 → 外部芯片初始化 → 读取硬件版本
3  → 创建所有Driver实例 → 创建所有Service实例
4  → 选择策略（观测器/死区/前馈/指令处理）
5  → 依赖注入到MotorCore → 配置中断优先级
6  → 初始化PB8为I2C → 启动TIM_FOC中断
7  → 启动RTOS调度器 → 通信协议栈启动 → running
8
9  running中：
10  IDLE → 参数修改 → 提交 → IDLE
11  IDLE → 整定 → IDLE
12  ENABLED → 故障 → FAULT → IDLE
```

不需要： new/delete、对象池、引用计数、create/destroy 模式。

11. 版本管理

三层版本体系：

代码块

```
1  BL + OTA 策略版本（联合）： 3.0
2  Bootloader本体、元数据结构、OTA包格式、分页协议、回滚策略
3  ← 这个变了 → APP 必须兼容
4
5  APP 版本： 1.2.0
6  电机驱动框架、FOC算法、CANopen OD、控制策略
7  ← 这个变了 → BL 不关心
8
```

12. 配置管理

12.1 配置分层

配置类型	位置	修改方式
硬件引脚映射	platform/board_config.h	编译期宏
驱动选择	config/build_config.h	编译期宏
策略选择	config/build_config.h	编译期宏
电机参数	config/motors/xxx.h	离线辨识后更新
PI 参数	config/motors/xxx.h	调参后更新
运行时指令	ControlCommand	SDO/CLI 写入
故障阈值	config/motors/xxx.h	编译期或 CALIBRATION 下修改

12.2 编译期开关宏

代码块

```
1 // config/build_config.h
2 #define ENCODER_TYPE          ENCODER_AS5047
3 #define SENSOR_TYPE          SENSOR_OPAMP_INTERNAL
4 #define PWM_TYPE              PWM_TIM1_TIM8_DUAL
5 #define MOTOR_COUNT           2
6 #define OBSERVER_TYPE         OBSERVER_ENCODER
7 #define DEADBAND_TYPE         DEADBAND_SIGN
8 #define FF_TYPE               FF_NONE
9 #define PROTO_TYPE            PROTO_CANOPEN
10 #define DEBUG_RTT_ENABLE      0
11 #define DEBUG_CLI_ENABLE      0
12 #define LOG_LEVEL             LOG_LEVEL_WARN
```

12.3 电机参数文件

```

1 // config/motors/luggage_motor_a.h
2 struct MotorParams {
3     uint8_t pole_pairs      = 4;
4     float   rated_current   = 2.0f;
5     float   Rs               = 0.85f;
6     float   Ld               = 0.0042f;
7     float   Lq               = 0.0042f;
8     float   flux_linkage     = 0.012f;
9     float   current_kp       = 8.4f;
10    float   current_ki       = 1700.0f;
11    float   speed_kp          = 0.01f;
12    float   speed_ki          = 0.2f;
13    float   dc_link_voltage   = 24.0f;
14    float   overcurrent_threshold = 4.0f;
15 };

```

12.4 参数修改权限

- **IDLE / SETUP:** 可编辑 pending_params，commit 后生效
- **ENABLED / FAULT / EMERGENCY:** 参数只读，禁止修改

13. C/C++ 边界约定

铁律

代码块

```

1  ISR 入口      → extern "C"
2  HAL 回调      → extern "C"
3  FreeRTOS 任务 → extern "C"
4  其余一切      → C++ (.cpp)

```

编译选项

代码块

```

1  -std=c++17
2  -fno-exceptions
3  -fno-rtti
4  -fno-use-cxa-atexit
5  -fno-threadsafe-statics
6  -Os

```

语言特性清单

✓ 使用: `enum class`, `constexpr`, `static_assert`, `nullptr`, `override`, `namespace`, `auto`, `if constexpr`, `std::array`, `std::atomic`, 模板, 虚函数

✗ 禁止: `std::vector/map/string`, 智能指针, `new/delete`, `iostream`, `throw/try/catch`, `dynamic_cast`

14. 编译与工具链

- **编译器:** ARM Compiler 6 (Clang 14 级别), C++17 大部分特性可用
- **IDE:** Keil MDK (编译调试) + VSCode (代码编辑, 可选)
- **调试:** SEGGER RTT (编译开关 `DEBUG_RTT_ENABLE`)、DWT Profiling (`DEBUG_DWT_ENABLE`)
- **日志等级:** NONE / ERROR / WARN / INFO / TRACE, 编译期宏控制

15. 文件目录结构

代码块

```

1  firmware/
2  |—— hal/                # ST HAL (CubeMX生成, 不改)
3  |—— driver/             # 驱动层 (按功能域分)
4  |   |—— encoder/       #   编码器: i_encoder.h, as5047.h/.cpp
5  |   |—— current_sense/  #   电流采样: i_current_sense.h,
   |   opamp_internal.h/.cpp
6  |   |—— gate_driver/    #   栅极驱动: i_gate_driver.h, fd6288.h/.cpp
7  |   |—— power_stage/    #   功率级: i_pwm_timer.h, tim1_tim8_dual.h/.cpp
8  |   |—— storage/        #   存储: w25qxx.h/.cpp
9  |   |—— display/        #   显示: oled_ssd1306.h/.cpp
10 |   |—— comm/           #   通信: fdcan_drv.h/.cpp, uart_drv.h/.cpp
11 |   |—— adc/            #   ADC: injected_3phase.h/.cpp
12 |—— platform/          # 平台层
13 |   |—— board_config.h  #   引脚映射
14 |   |—— product_luggage.h #   产品定义
15 |—— service/           # 业务层 (按功能域分)
16 |   |—— motor/          #   FOC算法: motor_core, observer_*, ff_*,
   |   deadband_*
17 |   |—— comms/          #   通信: canopen_service, cli_service, shared_data
18 |   |—— ota/            #   OTA: ota_service
19 |   |—— calibration/    #   标定: encoder_zero, current_offset

```

```
20 | | fault/          # 故障: fault_detector, fault_handler
21 | | debug/         # 调试: debug_bridge, dwt_profiler
22 | | app/           # 应用层
23 | | | main.cpp      # 唯一组装入口
24 | | | app_lifecycle.cpp # 初始化序列
25 | | | system_mode_manager.cpp
26 | | config/        # 配置
27 | | | build_config.h  # 编译期开关
28 | | | system_config.h # 系统参数
29 | | | motors/        # 电机参数库
30 | | | luggage_motor_a.h
```

16. 代码量估算

区域	预估行数
Driver 层 (所有功能域)	~1650
Service/motor (FOC+观测器+死区+前馈+PI+SVPWM+参数辨识+模式管理)	~2900
Service/comms (CANopen+CLI+共享数据)	~750
Service/OTA	~1500
Service/calibration + fault	~500
Platform + App + Config	~800
总计	~8000 - 10000

单人完全可维护。

17. 渐进式迁移路线

第一阶段：定接口（2-3天）

写好所有 `.h` 接口文件，编译通过但函数体为空。

第二阶段：迁移现有代码（1-2周）

按依赖顺序：电流采样 → 编码器 → PWM → FOC核心 → CANopen → OTA。每个模块迁移后单独验证。

第三阶段：加新策略（持续）

新建文件实现接口，改一行宏选型。不动框架代码。

18. 维护纪律

纪律一：接口一旦定义，不加字段

定义接口时多想一步"以后可能需要什么"。如果真需要加，新建一个继承接口，不改原接口。

纪律二：新增策略 = 新增文件，不改已有文件

加 SMO → 新建 `observer_smo.cpp`，改 `build_config.h` 一行宏。不动 `motor_core.cpp`。

纪律三：MotorCore::tick() 只调接口，不引入类型判断

`tick()` 里永远只有 `observer_->getTheta()`，不会出现 `if (type == SMO) {...}`。

19. 架构评估

维度	评分	说明
分层清晰度	9/10	五层单向依赖，域解耦明确
扩展性	9/10	四个纯虚扩展点，加策略不改框架
可维护性	8/10	函数短(≤50行)，单人能读懂全部
可移植性	8/10	换定时器/编码器/策略只改配置
过度设计风险	2/10	无注册中心、无动态分配、无多层继承——恰到好处
毕业前重构风险	1/10	接口稳定 + 三条纪律保证

结论： 本架构面向单人维护的电机驱动固件，通过五层单向依赖 + 两域解耦 + 四个纯虚扩展点，实现了可替换、可扩展、可移植、可维护的设计目标。所有设计模式均使用最简单的形式，不做过度抽象。代码量控制在 8000-10000 行，C++17 无堆分配，虚函数开销在 170MHz Cortex-M4 上可忽略。

下一步： 开始写接口头文件，进入第一阶段迁移。

需要我帮你生成一份**可直接复制到Word的纯文本版本**，并调整格式以适配Word的自动目录功能吗？

（注：文档部分内容可能由 AI 生成）